

Clase Teórica : Seguridad

Tomás F. Melli

October 2025

Índice

1	Introducción	2
1.1	Orígenes históricos	2
1.2	Formalización teórica	2
2	Marco de Trabajo: Conceptos Fundamentales	2
2.1	Propiedades de la Seguridad	2
2.2	Protocolos y Capas	3
3	Introducción a la Criptografía	4
3.1	Definiciones básicas	4
3.1.1	Criptología	5
3.2	Métodos Básicos de Cifrado	5
3.2.1	Cifrado por Sustitución	5
3.2.2	Cifrado por Transposición	6
3.3	Principio de Kerckhoffs	7
3.4	Longitud de la Clave y Factor de Trabajo	7
3.5	One-Time Pads (Rellenos de Una Sola Vez)	7
3.6	Cifrado de Bloque Iterativo	8
3.7	DES (Data Encryption Standard)	8
3.8	3DES (Triple DES)	9
4	Criptografía de Clave Simétrica (Cifrado por Clave Privada)	10
5	Criptografía de Clave Asimétrica (Criptografía de Clave Pública)	11
5.1	Propiedades y usos principales	12
5.2	Requisitos de seguridad	12
5.3	Interacción con la criptografía simétrica	12
5.4	RSA – Rivest–Shamir–Adleman	12
6	Firma Digital con Criptografía Asimétrica	14
6.1	Proceso de Firma Digital	15
6.2	Algoritmos de Hash	15
7	Autenticación basada en Claves Compartidas	16
7.1	Protocolo Challenge-Response de dos vías	16
7.2	Protocolo simplificado	16
7.3	Ataques por sesiones múltiples	17
7.4	Reglas de diseño	17
7.5	Distribución de Claves Confiable (KDC - Key Distribution Center)	17
8	Ataques de Red	18

1 Introducción

La **Seguridad en Redes** se apoya en los principios de la criptografía, disciplina fundamental para garantizar la confidencialidad, integridad, autenticación y no repudio de la información transmitida a través de sistemas de comunicación. Su desarrollo ha acompañado la evolución de las telecomunicaciones y la informática, siendo hoy un pilar esencial en el diseño de protocolos y arquitecturas seguras.

1.1 Orígenes históricos

Los primeros avances significativos en criptografía moderna se remontan a principios del siglo XX. En **1918**, los ingenieros alemanes **Arthur Scherbius** y **Richard Ritter** desarrollaron la máquina **Enigma**, un sistema electromecánico de cifrado que Alemania utilizó ampliamente durante la **Segunda Guerra Mundial**. El diseño de Enigma introdujo la idea de sistemas criptográficos basados en operaciones mecánicas complejas y configuraciones de clave variables, marcando el inicio de la era moderna de la criptografía.

1.2 Formalización teórica

El estudio formal de la criptografía como ciencia se consolidó en la década de 1940 gracias a los trabajos de **Claude E. Shannon**, considerado el padre de la teoría de la información. En **1949**, Shannon publicó en el *Bell System Technical Journal* su influyente artículo:

C. E. Shannon, "Communication Theory of Secrecy Systems", *The Bell System Technical Journal*, vol. 28, no. 4, pp. 656–715, Oct. 1949.

Este trabajo sentó las bases matemáticas de la criptografía moderna, definiendo conceptos como **seguridad perfecta** y **entropía de la información**, y estableciendo los fundamentos teóricos que permiten analizar la robustez de un sistema criptográfico frente a ataques.

De manera complementaria, Shannon había desarrollado en **1945** un *reporte interno* en los Bell Labs titulado "*A Mathematical Theory of Cryptography*", en el cual anticipaba muchas de las ideas que luego formalizaría en su publicación de 1949. Dicho informe marcó el inicio de una visión científica y sistemática de la criptografía, alejándola de su carácter artesanal y secreto que había predominado durante siglos.

2 Marco de Trabajo: Conceptos Fundamentales

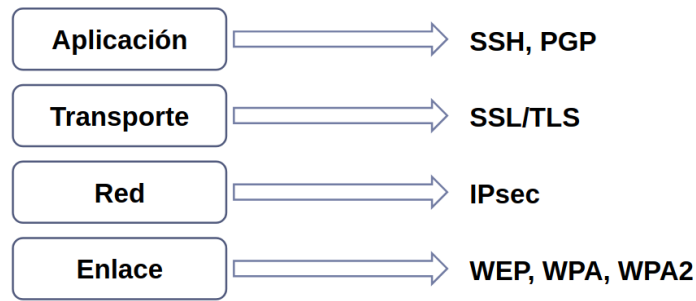
El estudio de la **Seguridad en Redes** se apoya en un conjunto de propiedades que definen los objetivos esenciales de la protección de la información. Cada tecnología o protocolo de seguridad debe ser evaluado en función del subconjunto de estas propiedades que logra garantizar.

2.1 Propiedades de la Seguridad

- **Confidencialidad:** Los mensajes sólo deben poder ser comprendidos por las partes autorizadas involucradas en la comunicación. Se busca impedir el acceso no autorizado a la información.
- **Integridad:** Los mensajes enviados no pueden ser alterados ni modificados durante su transmisión. Cualquier cambio debe poder ser detectado.
- **Originalidad:** El mensaje recibido debe ser genuino y no una copia artificial o repetición fraudulenta de un mensaje previo.
- **Temporalidad:** El mensaje no debe haber sido demorado o retenido maliciosamente, asegurando que la comunicación ocurra dentro de un marco temporal esperado.
- **Autenticación:** Cada parte participante debe poder verificar la identidad de la otra. Ninguna entidad debe poder suplantar la identidad de otra de manera no autorizada.
- **Control de acceso (Autorización):** Solo determinados usuarios o sistemas remotos pueden realizar ciertas acciones o acceder a determinados recursos.
- **Disponibilidad:** Todo usuario legítimo debe tener la posibilidad de acceder a los recursos del sistema y ser eventualmente admitido para utilizarlos.
- **No repudio:** Ninguna de las partes involucradas puede negar haber participado en una comunicación o transacción una vez efectuada.

Ante cada tecnología o protocolo de seguridad informática, es fundamental preguntarse: *¿Qué subconjunto de estas propiedades provee?*

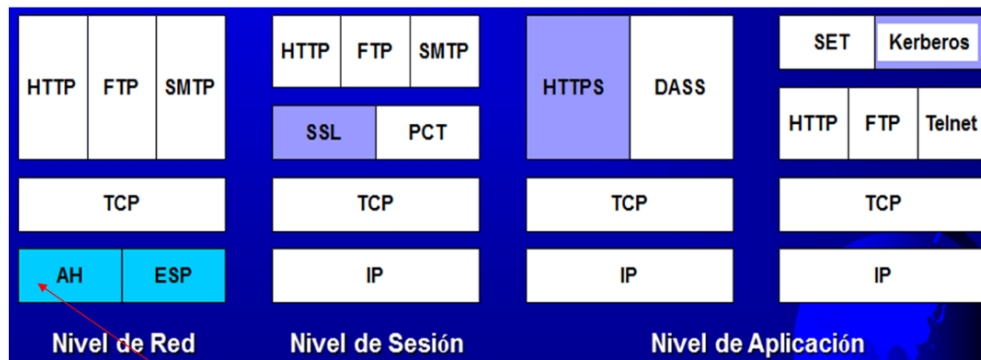
2.2 Protocolos y Capas



La seguridad puede implementarse en distintas capas del modelo de comunicación en red, cada una orientada a resolver problemas específicos. Los principales mecanismos incluyen:

- **Capa de Enlace:** *WEP, WPA, WPA2* – proporcionan mecanismos de cifrado y autenticación en redes inalámbricas.
- **Capa de Red:** *IPsec* – conjunto de protocolos que proveen servicios de seguridad directamente sobre IP.
- **Capa de Transporte:** *SSL/TLS* – garantizan comunicaciones seguras entre aplicaciones cliente-servidor.
- **Capa de Aplicación:** *SSH, PGP* – protegen el acceso remoto y el intercambio de correos o archivos con cifrado y autenticación.

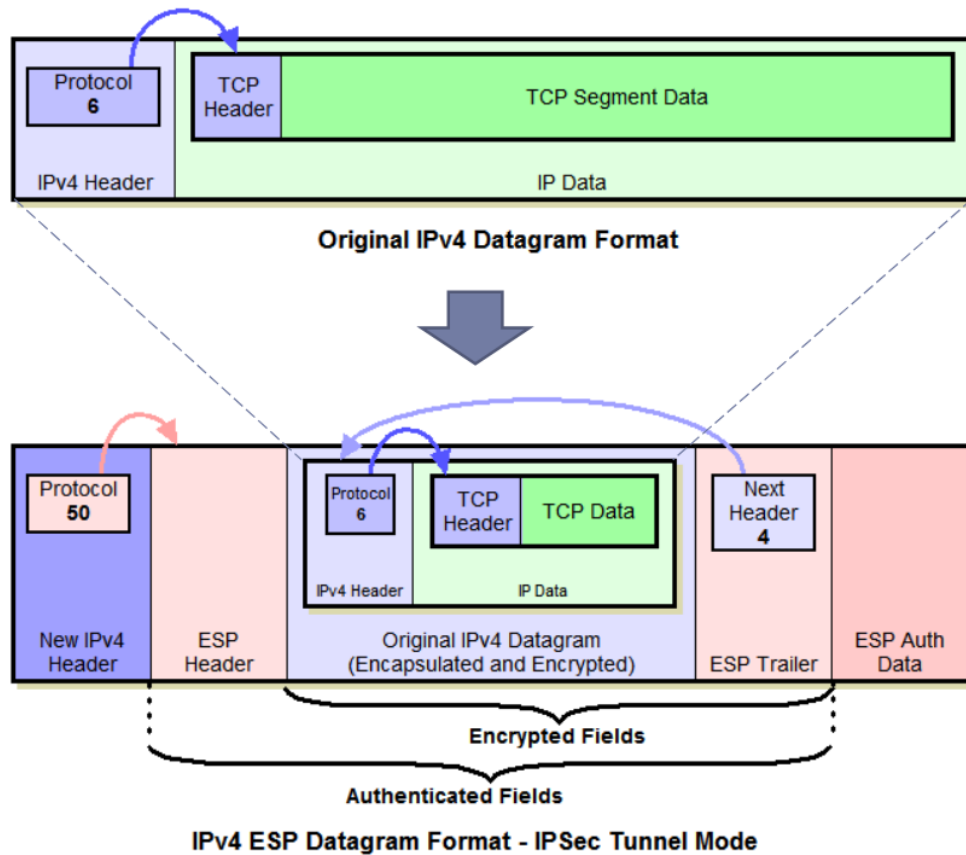
Seguridad en las Capas de Red



Como ejemplo, el conjunto de protocolos **IPsec** implementa seguridad directamente en la capa de red, permitiendo comunicaciones seguras entre hosts o redes mediante dos mecanismos principales:

- **Authentication Header (AH):** Proporciona integridad, autenticación y no repudio de los paquetes IP.
- **Encapsulating Security Payload (ESP):** Proporciona confidencialidad mediante el cifrado del contenido de los paquetes.

Así, IPsec integra funciones de protección en la propia infraestructura de red, permitiendo construir canales seguros (túneles) que garantizan múltiples propiedades de seguridad simultáneamente.



3 Introducción a la Criptografía

El término **criptografía** proviene del griego antiguo: (*krypto*), que significa “oculto”, y (*graphos*), que significa “escribir”. Literalmente, **criptografía** significa “*escritura oculta*”.

Un recorrido rápido

La criptografía moderna abarca un conjunto amplio de técnicas y mecanismos matemáticos destinados a proteger la información. Entre los temas fundamentales se encuentran:

- Generación de números al azar.
- Hashing (funciones resumen o de dispersión).
- Criptografía de **claves simétricas**.
- Criptografía de **claves asimétricas**.

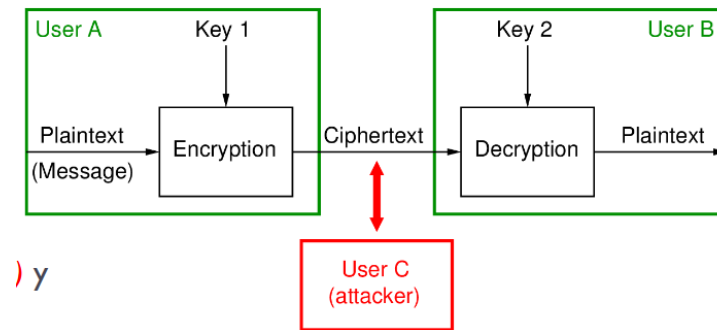
3.1 Definiciones básicas

- **Cifrado:** Consiste en una transformación carácter por carácter, o bit por bit, que convierte un mensaje en una forma ilegible. Esta operación no depende de la estructura lingüística del mensaje original.
- **Código:** En este caso, se reemplazan palabras o frases completas por otras palabras, símbolos o expresiones equivalentes. Un ejemplo histórico es el **código basado en el idioma indígena Navajo**, utilizado por las fuerzas estadounidenses en el Pacífico durante la Segunda Guerra Mundial. Los llamados *code talkers* Navajos operaban a ambos lados de la línea de comunicación, empleando un idioma oral sin reglas gramaticales formales, poco conocido y no escrito, lo que lo hacía extremadamente difícil de descifrar.

Referencias audiovisuales:

- [Fragmento histórico sobre los code talkers \(YouTube\)](#)
- [Documental adicional sobre la criptografía Navajo \(YouTube\)](#)

Conceptos fundamentales



- Los mensajes que se desean proteger se denominan **texto plano** o *plaintext*.
- Dichos mensajes se procesan mediante una **función o algoritmo de cifrado**, el cual está **parametrizado por una clave**.
- El resultado del proceso se conoce como **texto cifrado** o *ciphertext*, y representa los datos a transmitir por el canal.
- Se asume que un intruso puede escuchar y copiar completamente el ciphertext transmitido (es decir, un *intruso pasivo*).

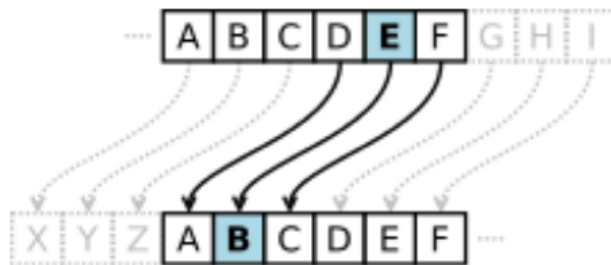
3.1.1 Criptología

El campo general que abarca tanto la creación de sistemas de cifrado (**criptografía**) como su análisis y ruptura (**criptoanálisis**) se denomina **criptología**. Esta disciplina combina fundamentos matemáticos, computacionales y estadísticos, constituyendo uno de los pilares de la seguridad de la información moderna.

3.2 Métodos Básicos de Cifrado

Los métodos clásicos de cifrado se basan principalmente en dos grandes familias de transformaciones: **sustitución** y **transposición**. Ambos mecanismos buscan alterar el mensaje original (texto plano) para dificultar su interpretación por terceros no autorizados.

3.2.1 Cifrado por Sustitución



En este método, cada letra o grupo de letras del texto plano se reemplaza por otra letra o grupo de letras con el objetivo de enmascarar el contenido. Es importante notar que, aunque los símbolos cambian, el **orden del texto plano se preserva**.

Ejemplo clásico:

- **Cifrado del César** (Julius Caesar, 50 a.C): Consiste en una rotación del alfabeto por una cantidad fija de posiciones. Por ejemplo, un desplazamiento de 3 convierte la A en D, la B en E, y así sucesivamente.
- **Sustitución monoalfabética**: Cada letra se mapea con otra cualquiera de forma fija. Dado que existen $26! \approx 4 \times 10^{26}$ posibles permutaciones del alfabeto, el número de llaves posibles es astronómico.

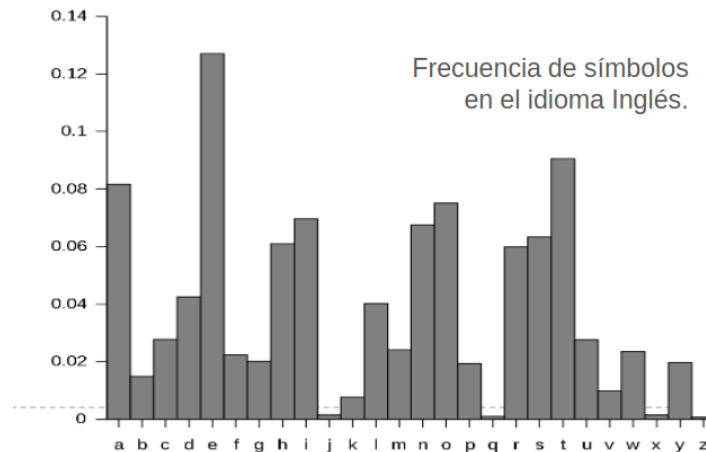
Ejemplo de sustitución monoalfabética:

texto plano: a b c d e f g h i j k l m n o p q r s t u v w x y z
texto cifrado: Q W E R T Y U I O P A S D F G H J K L Z X C V B N M

Aunque el número de combinaciones posibles es enorme, este tipo de cifrados puede ser **quebrado fácilmente** mediante análisis de frecuencia, aprovechando las propiedades estadísticas de los lenguajes naturales.

Extensión: Sustitución Polialfabética En este caso, se utiliza un alfabeto distinto para cada letra del mensaje. De este modo, la frecuencia de aparición de símbolos deja de ser constante, haciendo más difícil el criptoanálisis. Sin embargo, aun así pueden aplicarse técnicas estadísticas más avanzadas (como el análisis de Kasiski o la prueba de Friedman).

Frecuencia de símbolos en el idioma inglés: Las letras más comunes son E, T, A, O, I, N, S, H, R, D, L, U.



3.2.2 Cifrado por Transposición

M	E	G	A	B	U	C	K	Plaintext
7	4	5	1	2	8	3	6	pleasetransferonemilliondollarsto
p	i	e	a	s	e	t	r	myswissbankaccountsixtwo
a	n	s	f	e	r	o	n	
e	m	i	l	l	i	o	n	Ciphertext
d	o	l	l	a	r	s	t	AFLLSKSOSELAWAIATOSSCTCLNMOMANT
o	m	y	s	w	i	s	s	ESILYNTWRNNTSOWDPAEDOBUEIRICXB
b	a	n	k	a	c	c	o	
u	n	t	s	i	x	t	w	
o	t	w	o	a	b	c	d	

En este tipo de cifrado no se sustituyen los caracteres del texto plano, sino que se **reordena su posición**. El contenido de las letras permanece igual, pero el orden cambia, lo cual también produce confusión en el mensaje.

Procedimiento típico:

- Se elige una palabra clave sin letras repetidas, por ejemplo: MEGABUCK.
- La clave se utiliza para **numerar las columnas**, asignando el número 1 a la letra más próxima al inicio del alfabeto, 2 a la siguiente, y así sucesivamente.
- El texto plano se escribe horizontalmente en filas, completando la matriz si es necesario.
- El texto cifrado se obtiene leyendo **por columnas**, comenzando por la columna cuya letra clave sea la menor alfabéticamente.

Este método mantiene las letras originales del mensaje pero cambia su orden, lo que genera una apariencia desorganizada y difícil de interpretar sin la clave correcta.

3.3 Principio de Kerckhoffs

El **Principio de Kerckhoffs** establece que:

“Todos los algoritmos deben ser públicos, sólo las claves deben ser secretas.”

Este principio fue enunciado por **Auguste Kerckhoffs** en 1883 en su artículo:

- **KERCKHOFF, A.** *La Cryptographie Militaire*.

La idea central es que mantener secreto un algoritmo raramente funciona, y depender de la “*seguridad por desconocimiento*” es peligroso. Por ello, la mayoría de los sistemas modernos de criptografía aplican este principio: los algoritmos son públicos y la seguridad depende únicamente de la clave secreta.

3.4 Longitud de la Clave y Factor de Trabajo

Cuando la seguridad reside en la clave, su **longitud** se convierte en un factor crítico de diseño. Algunos ejemplos:

- Clave de 2 dígitos decimales: 100 combinaciones posibles.
- Clave de 6 dígitos decimales: 1 millón de combinaciones.

Factor de Trabajo: tiempo o esfuerzo requerido por un criptoanalista para descubrir la clave. Este factor crece **exponencialmente** con la longitud de la clave. Por lo tanto, un algoritmo robusto combinado con una clave suficientemente larga hace que romper la seguridad sea impráctico.

Recomendaciones generales de longitud de clave:

- 64 bits: protege contra ataques básicos, por ejemplo, la lectura de correos por parte de usuarios ocasionales o menores.
- 128-256 bits: estándar para uso comercial rutinario.
- Más de 256 bits: recomendado para sistemas con necesidades críticas de seguridad.

3.5 One-Time Pads (Rellenos de Una Sola Vez)

Los **One-Time Pads** constituyen un método de cifrado teóricamente **inviolable** cuando se aplican correctamente. Sus características principales son:

- Se elige una cadena de bits aleatoria (*random bit string*) de **igual longitud que el texto plano**.
- Se aplica la operación **XOR** bit a bit entre el texto plano y la clave para encriptar y desencriptar.
- Garantiza seguridad absoluta: “*immune a todos los ataques actuales y futuros, sin importar la potencia computacional del intruso*”. Este resultado se fundamenta en la **Teoría de la Información** y fue probado por Claude Shannon en 1945/1949.
- Principal limitación: la distribución y protección de la clave es compleja, lo que hace que su uso sea poco práctico en la mayoría de las aplicaciones.

Esquema

Mensaje 1:	1001001 0100000 1101100 1101111 1110110 1100101 0100000 1111001 1101111 1110101 0101110
Relleno 1:	1010010 1001011 1110010 1010101 1010010 1100011 0001011 0101010 1010111 1100110 0101011
Texto cifrado:	0011011 1101011 0011110 0111010 0100100 0000110 0101011 1010011 0111000 0010011 0000101

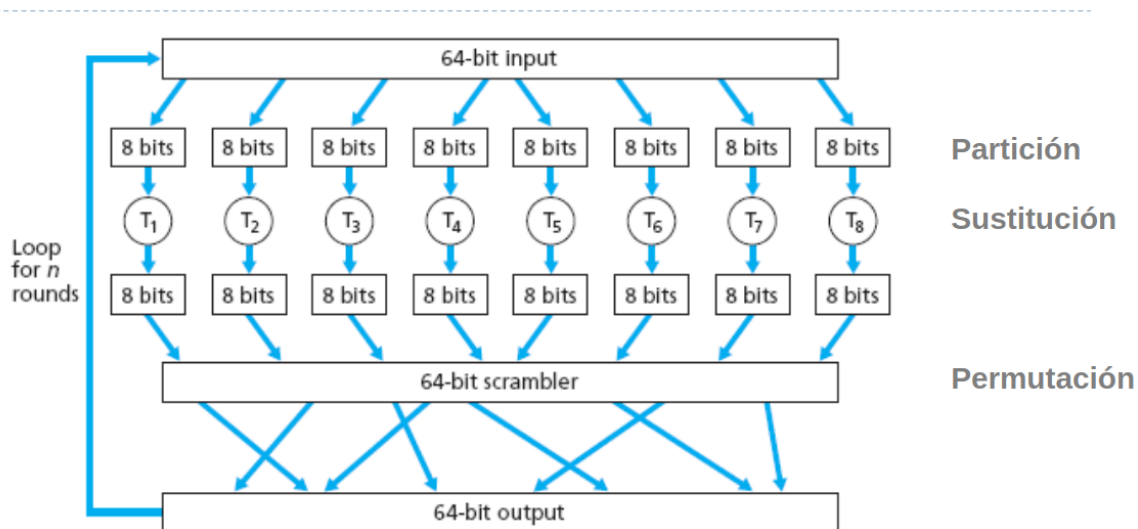
3.6 Cifrado de Bloque Iterativo

En el **cifrado de bloque**, se toma un **bloque de n bits** del texto plano y se transforma utilizando la clave para producir un **bloque de n bits de texto cifrado**. Este proceso se repite de manera iterativa sobre todos los bloques del mensaje.

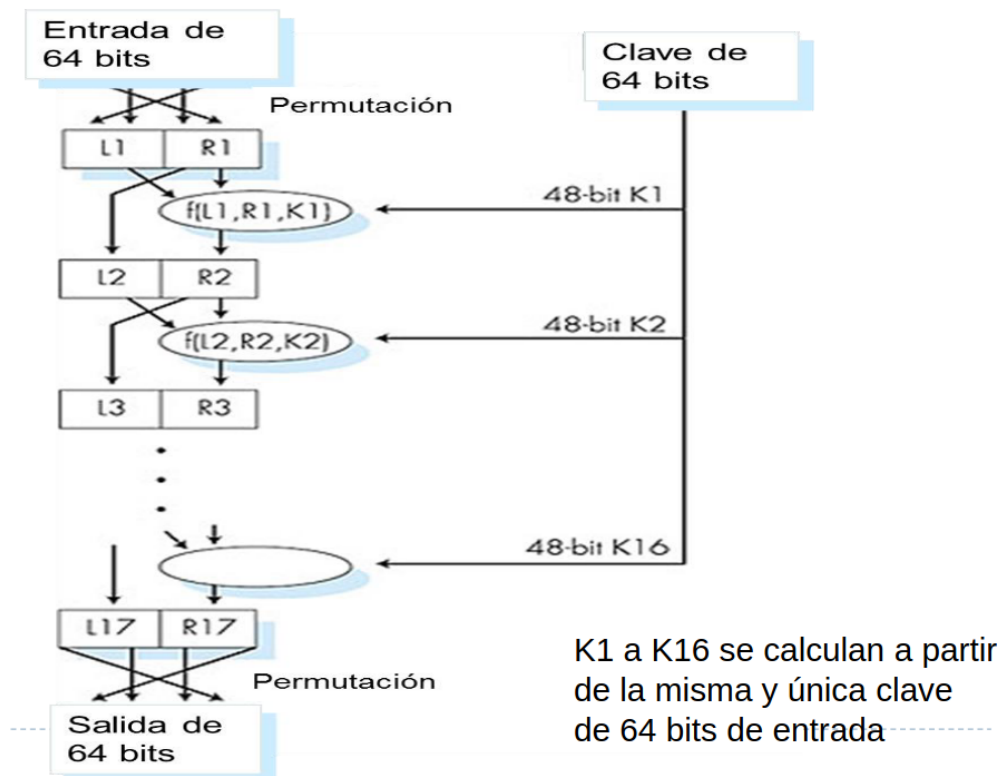
- Referencia: *Computer Networking: Top-Down Approach*, 6ta edición, pág. 680.
- Las técnicas de cifrado de bloque combinan operaciones de:
 - **Sustitución** – reemplazo de bits o sub-bloques por otros valores.
 - **Permutación** – reordenamiento de los bits dentro del bloque.
 - **Partición** – división del bloque en sub-bloques que se procesan parcialmente.

Este enfoque permite construir algoritmos seguros y eficientes para cifrado de mensajes de cualquier longitud, siendo la base de los algoritmos de cifrado modernos como DES, AES y sus variantes.

Esquema



3.7 DES (Data Encryption Standard)



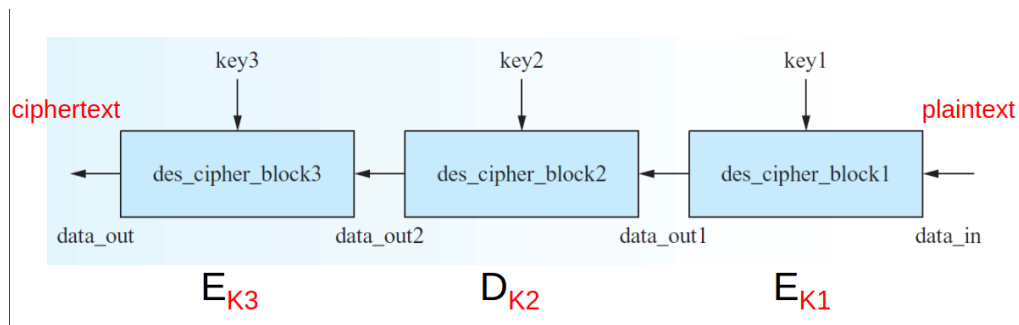
El **Data Encryption Standard (DES)**, desarrollado por IBM en 1977, es uno de los primeros algoritmos de cifrado de bloques ampliamente utilizados. Sus características principales son:

- Basado en **cifrado de bloques**, operando sobre bloques de **64 bits**.
- Trabaja a nivel de **bits**, no de caracteres, y combina técnicas de **sustitución y transposición**.
- Cada bloque de 64 bits pasa por $n = 16$ iteraciones, cada una usando una subclave derivada de la clave original.
- La longitud de la clave original es de 64 bits, de los cuales 56 bits son efectivos y 8 bits se utilizan como paridad.

Limitaciones y Vulnerabilidades

- Con el tiempo, el tamaño de la clave de DES se volvió insuficiente, haciendo que los ataques de fuerza bruta fueran cada vez más rápidos y prácticos.
- Se identificaron vulnerabilidades para ciertas claves específicas.
- Hoy en día, se recomienda el uso de **AES** como mínimo estándar de cifrado.
- Incrementar la longitud de la clave en variantes de DES ayuda a mitigar estas vulnerabilidades.

3.8 3DES (Triple DES)



Para aumentar la seguridad de DES, se desarrolló **Triple DES (3DES)**, que aplica el algoritmo DES tres veces con diferentes claves:

- **Encriptación:**

$$\text{ciphertext} = E_{k3}(D_{k2}(E_{k1}(\text{plaintext})))$$

- **Desencriptación:**

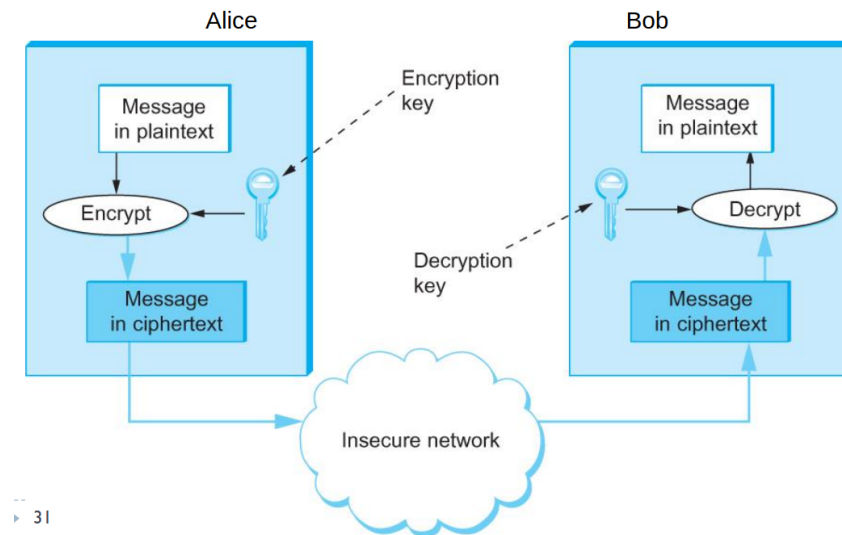
$$\text{plaintext} = D_{k1}(E_{k2}(D_{k3}(\text{ciphertext})))$$

- Las subclaves K_1 a K_{16} se calculan a partir de cada clave individual de 64 bits.

4 Criptografía de Clave Simétrica (Cifrado por Clave Privada)

La **criptografía simétrica** es un modelo de cifrado en el que el mismo secreto (clave) es compartido entre el emisor y el receptor. Este tipo de algoritmos se denomina también **cifrado por clave privada**.

Esquema



Sea P el **plaintext** (mensaje original) y K la **clave secreta compartida**. El cifrado y descifrado se definen mediante funciones matemáticas:

- **Encriptación:**

$$C = E_K(P)$$

donde C es el **ciphertext** generado al aplicar el algoritmo E con la clave K sobre P .

- **Desencriptación:**

$$P = D_K(C)$$

que recupera el mensaje original usando la misma clave K .

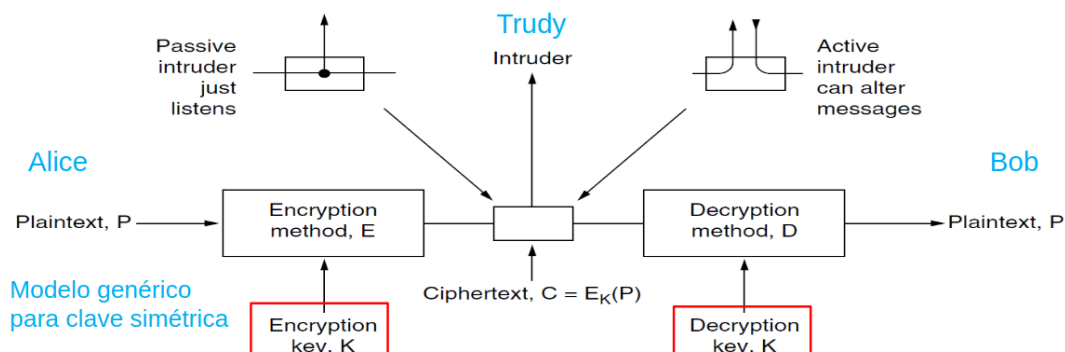
- De esta forma, se cumple la relación fundamental:

$$P = D_K(E_K(P))$$

Característica Principal

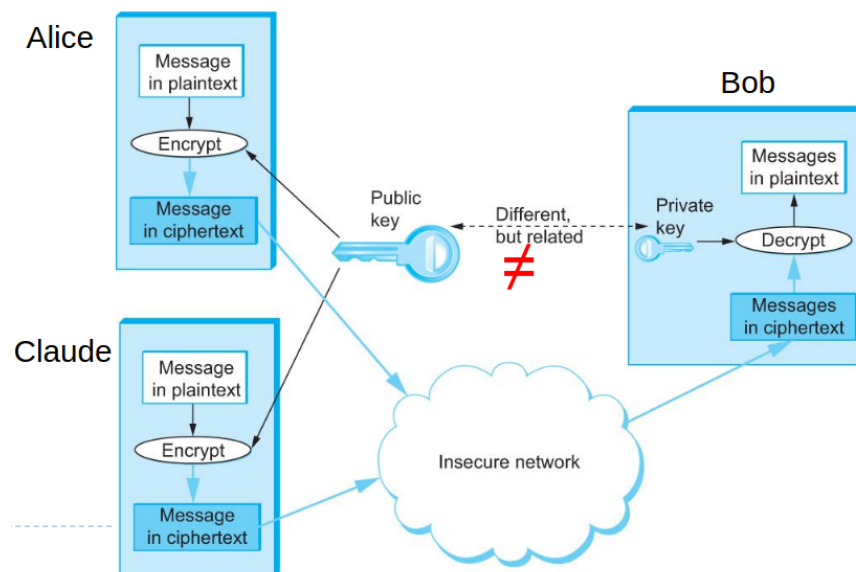
El algoritmo de encriptación E y el de desencriptación D son **funciones matemáticas dependientes de dos parámetros**, uno de los cuales es la clave secreta K . Este modelo asegura que solo quienes poseen la clave pueden acceder al contenido del mensaje.

Representación Conceptual



Alice y **Bob** comparten la clave K , mientras que **Trudy** representa un atacante potencial que intenta interceptar el mensaje cifrado sin conocer K .

5 Criptografía de Clave Asimétrica (Criptografía de Clave Pública)



La **criptografía de clave asimétrica** —también conocida como **criptografía de clave pública**— ofrece un enfoque radicalmente distinto al de la criptografía simétrica. Mientras que en la criptografía simétrica el emisor y el receptor deben compartir una misma clave secreta, la criptografía de clave pública evita la necesidad de un acuerdo previo sobre una clave secreta compartida (problema que resulta crítico cuando las partes nunca se han encontrado).

Historia

Los desarrollos clave en este área incluyen el trabajo de **Diffie–Hellman** (1976), que introdujo el concepto de intercambio de claves de forma segura, y el esquema **RSA** (Rivest–Shamir–Adleman, 1978), que proporciona un método práctico de cifrado y firma basados en la factorización de enteros.

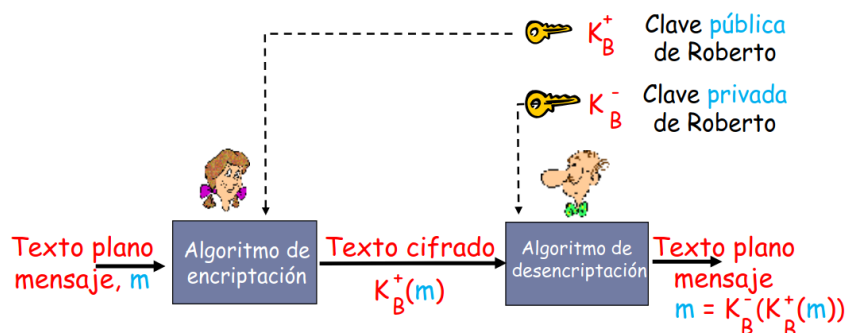
Idea fundamental

En criptografía de clave pública cada entidad posee dos claves relacionadas:

- Una **clave pública** (K^+): conocida por cualquiera, usada para cifrar mensajes dirigidos al propietario o para verificar firmas.
- Una **clave privada** (K^-): conocida sólo por el propietario, usada para descifrar mensajes o para generar firmas digitales.

La propiedad esencial es que, aunque las dos claves están matemáticamente relacionadas, es **computacionalmente inviable** (con los parámetros apropiados) obtener la clave privada a partir de la clave pública.

Esquema genérico



Si Bob tiene par de claves (K_B^+, K_B^-) , entonces para enviar un mensaje m de forma confidencial a Bob se utiliza:

$$\text{ciphertext} = E_{K_B^+}(m)$$

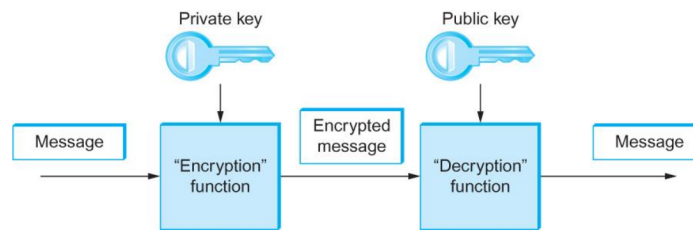
y Bob recupera el texto original con:

$$m = D_{K_B^-}(E_{K_B^+}(m)).$$

Aquí E y D representan los algoritmos de encriptación y descryptación respectivamente; la seguridad radica en la dificultad de obtener K_B^- a partir de K_B^+ .

5.1 Propiedades y usos principales

- **Confidencialidad:** Cualquiera puede cifrar para el propietario usando su clave pública, pero sólo el propietario puede descifrar con su clave privada.
- **Autenticación y firma digital:** El propietario puede firmar con su clave privada y cualquiera puede verificar la firma con la clave pública; esto proporciona autenticación e integridad (y no repudio cuando se combina con políticas adecuadas). Como se ve en este esquema :



- **Intercambio de claves:** Protocolos como Diffie–Hellman permiten acordar secretos compartidos entre dos partes a través de un canal inseguro, sirviendo como base para establecer claves simétricas para comunicación posterior.

5.2 Requisitos de seguridad

Para que un esquema de clave pública sea práctico y seguro, deben cumplirse condiciones como:

- Debe ser **eficiente** cifrar y descifrar con las claves correspondientes (dadas las claves correctas).
- Debe ser **computacionalmente intractable** derivar la clave privada a partir de:
 - la clave pública, o
 - un conjunto de pares (ciphertext, plaintext) conocidos.
- Los parámetros (tamaños de clave, curvas, módulos, etc.) deben elegirse para garantizar seguridad durante el horizonte temporal requerido frente a los recursos computacionales esperados de los atacantes.

5.3 Interacción con la criptografía simétrica

En la práctica, la criptografía de clave pública y la simétrica se usan **combinadas**: los esquemas de clave pública se utilizan para autenticar e intercambiar claves simétricas (o para cifrar pequeñas piezas de información), y las claves simétricas se utilizan para cifrar el flujo de datos por su eficiencia en grandes volúmenes.

5.4 RSA – Rivest–Shamir–Adleman

El algoritmo **RSA** es uno de los esquemas de criptografía de clave pública más utilizados. Su seguridad se basa en propiedades matemáticas relacionadas con la factorización de números primos grandes y la exponenciación modular.

Principios Teóricos

- Factorizar números muy grandes en primos es computacionalmente costoso.
- Existe una propiedad de conmutatividad en la exponenciación modular:

$$(x^a)^b \equiv (x^b)^a \pmod{n}$$

- RSA genera un par de claves que:
 1. No son derivables una de la otra de manera práctica.
 2. Son inversas matemáticamente: lo que se cifra con una puede descifrarse con la otra.

Generación de Claves

1. Elegir dos números primos grandes p y q (por ejemplo, de 1024 bits cada uno).
2. Calcular $n = p \cdot q$ y $z = (p - 1) \cdot (q - 1)$.
3. Elegir un entero e tal que $1 < e < z$ y que sea primo relativo con z .
4. Calcular d tal que $e \cdot d \equiv 1 \pmod{z}$.
5. La **clave pública** es (n, e) , y la **clave privada** es (n, d) .

Encriptación y Desencriptación

Dado un mensaje m (plaintext):

- **Encriptación:**

$$c = m^e \pmod{n}$$

donde c es el ciphertext.

- **Desencriptación:**

$$m = c^d \pmod{n}$$

recuperando el mensaje original.

Ejemplo Numérico

Plaintext (P)		Ciphertext (C)		After decryption	
Symbolic	Numeric	P^3	$P^3 \pmod{33}$	C^7	$C^7 \pmod{33}$
S	19	6859	28	13492928512	19
U	21	9261	21	1801088541	21
Z	26	17576	20	1280000000	26
A	01	1	1	1	01
N	14	2744	5	78125	14
N	14	2744	5	78125	14
E	05	125	26	8031810176	05
Sender's computation				Receiver's computation	

- $p = 3, q = 11 \Rightarrow n = 33, z = (p - 1)(q - 1) = 20$
- Elegimos $e = 3$, y calculamos $d = 7$ tal que $e \cdot d \equiv 1 \pmod{20}$
- Mensaje m se cifra como $c = m^3 \pmod{33}$ y se descifra como $m = c^7 \pmod{33}$

Propiedad Importante

RSA permite operar de manera inversa con las claves:

$$K_B^-(K_B^+(m)) = m \quad \text{o} \quad K_B^+(K_B^-(m)) = m$$

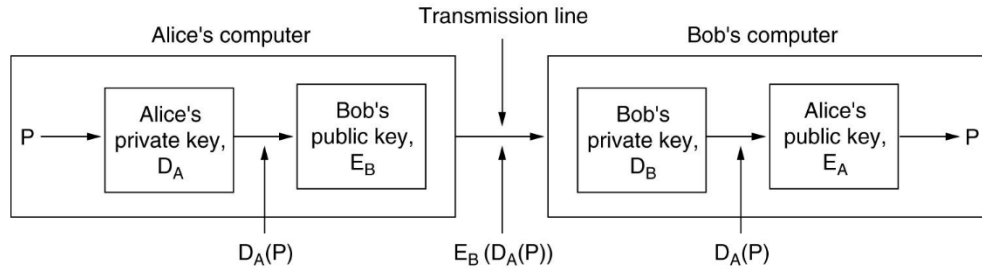
Esto significa que podemos:

- Usar primero la **clave pública** para cifrar y luego la **clave privada** para descifrar (confidencialidad).
- Usar primero la **clave privada** para "firmar" y luego la **clave pública** para verificar (firma digital y autenticación).

Esta propiedad es fundamental para los sistemas de **firma digital** y verificación de autenticidad.

6 Firma Digital con Criptografía Asimétrica

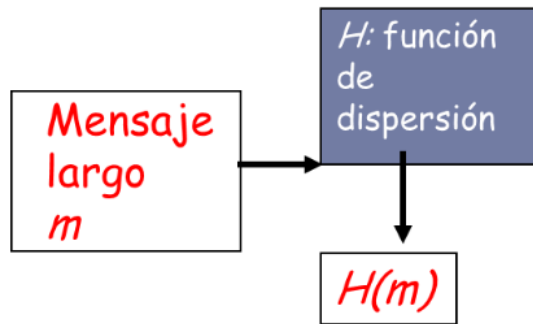
La **firma digital** es un mecanismo de seguridad que permite garantizar la **autenticidad**, **integridad** y **no repudio** de un mensaje mediante criptografía asimétrica.



- Sea un mensaje P que Alice desea enviar a Bob.
- Para garantizar que solo Alice pudo generar el mensaje y su firma, se aplica la **clave privada** de Alice para firmar el mensaje.
- Bob, utilizando la **clave pública** de Alice, puede verificar tanto la autoría como la integridad del mensaje.

Resumen del Mensaje – Message Digest

Encriptar mensajes completos con criptografía asimétrica es costoso computacionalmente. Por ello se utiliza una función de **hash** H para generar un **resumen de tamaño fijo** del mensaje:

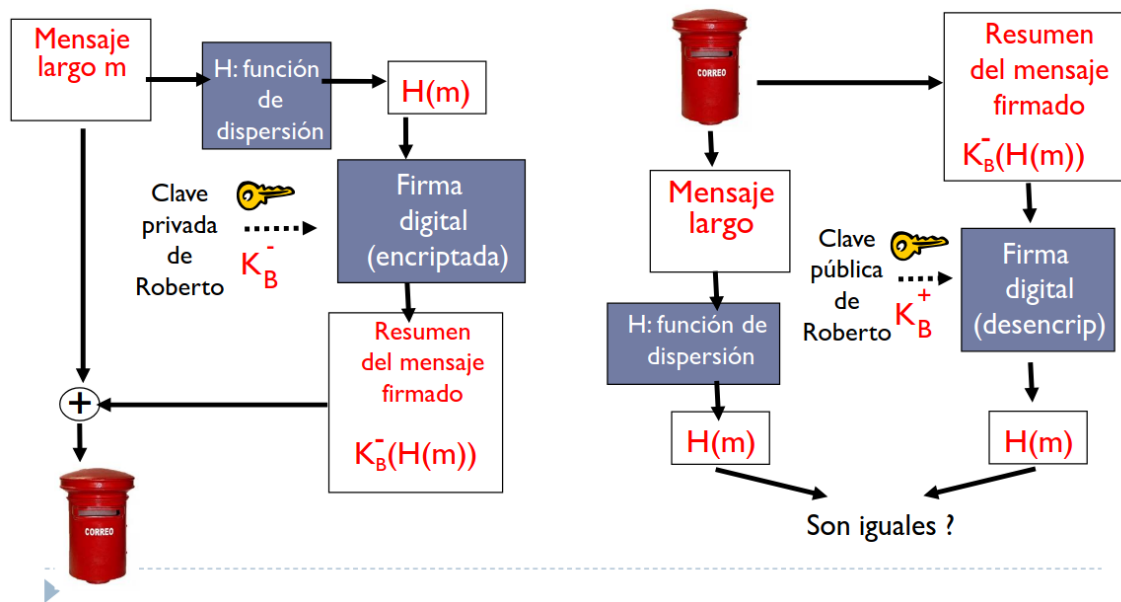


$$H : m \mapsto H(m)$$

Propiedades de la función de hash:

- Muchos a uno.
- Produce un resumen de tamaño fijo.
- Es computacionalmente inviable reconstruir m a partir de $H(m)$.

6.1 Proceso de Firma Digital

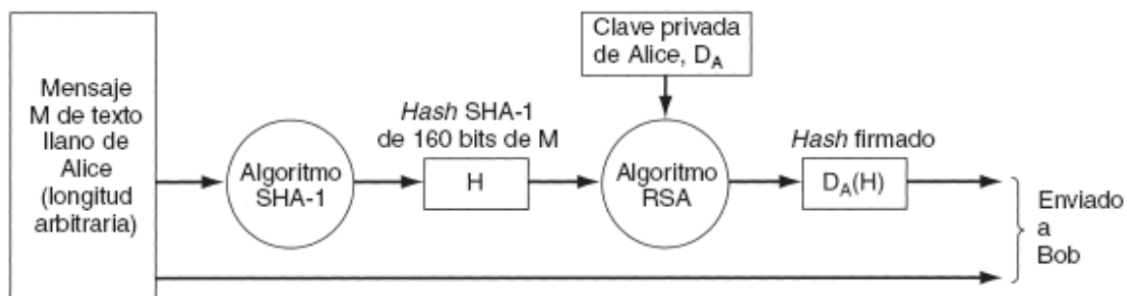


1. Alice genera un mensaje largo m que desea enviar a Bob.
2. Alice aplica la función de dispersión H al mensaje: $H(m)$.
3. Alice cifra el resumen $H(m)$ con su **clave privada**: Firma = $K_A^-(H(m))$.
4. Alice envía el mensaje m junto con la **firma digital** a Bob.
5. Bob recibe el mensaje y la firma, descifra la firma usando la **clave pública de Alice**: $K_A^+(Firma)$.
6. Bob compara el resultado del descifrado con $H(m)$ calculado a partir del mensaje recibido. Si coinciden, se verifica la integridad y autenticidad del mensaje.

6.2 Algoritmos de Hash

- **MD5 (RFC 1321)**: produce un resumen de 128 bits; colisiones conocidas.
- **SHA-1 (NIST, FIPS PUB 180-1)**: resumen de 160 bits; colisiones conocidas.
- **SHA-2**: resumen de 256 y 512 bits; mayor seguridad.

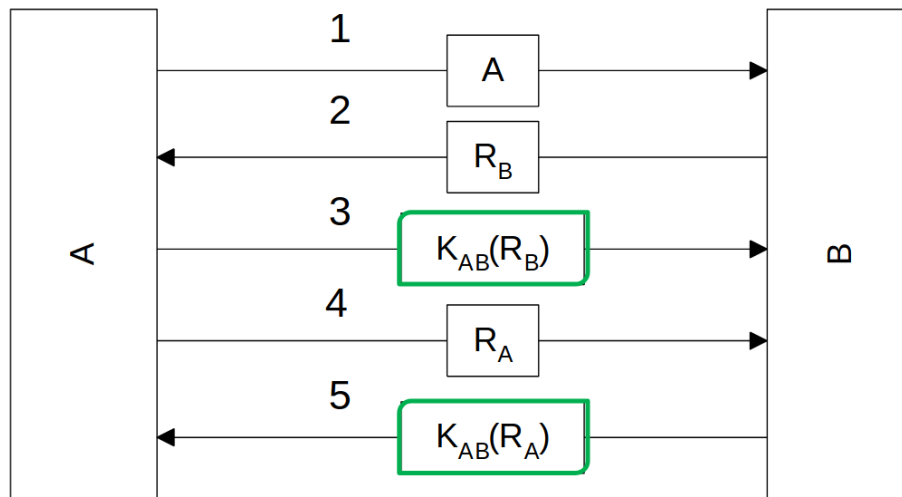
Firma Digital con RSA y SHA-1



El resumen del mensaje $H(m)$ se cifra con la clave privada de Alice para generar la firma digital. Luego, Bob utiliza la clave pública de Alice para descifrar la firma y compara el resultado con el hash del mensaje recibido para verificar integridad y autenticidad.

7 Autenticación basada en Claves Compartidas

7.1 Protocolo Challenge-Response de dos vías

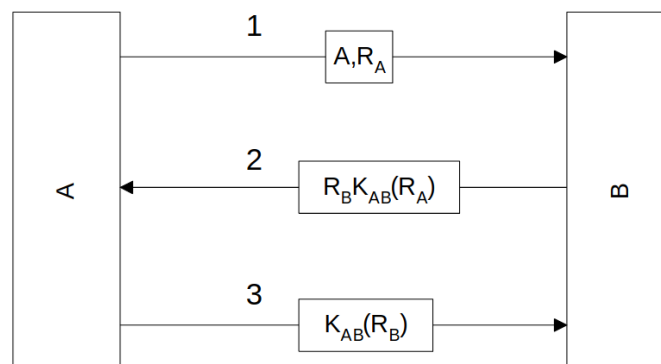


- K_{AB} : clave compartida entre A y B.
- R : número aleatorio (*random*) enviado sin encriptar.

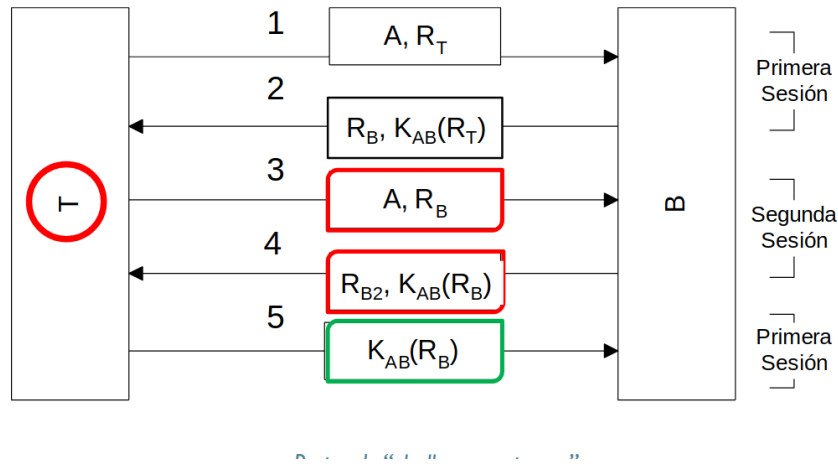
1. A envía su identidad a B.
2. Dado que B aún no puede determinar si el mensaje proviene realmente de A o de un tercero T, B elige un challenge R_B (número aleatorio suficientemente grande) y se lo envía a A sin encriptar.
3. A encripta el challenge recibido usando la clave compartida: $K_{AB}(R_B)$, y lo devuelve a B.
4. Cuando B recibe este texto cifrado, sabe inmediatamente que proviene de A, ya que solo A posee K_{AB} .
5. En este punto, B está seguro de la identidad de A, pero A aún no posee garantía sobre B.
6. Para verificar la identidad de B, A elige un número al azar R_A y se lo envía a B sin encriptar.
7. B responde con $K_{AB}(R_A)$, asegurando así la identidad de B.
8. Si ambos desean establecer una clave de sesión K_S , A selecciona K_S y se lo comunica a B cifrado con K_{AB} .

7.2 Protocolo simplificado

En la versión simplificada, cada participante transmite su identidad y el challenge elegido en el mismo mensaje, sin esperar la respuesta del otro participante.



7.3 Ataques por sesiones múltiples

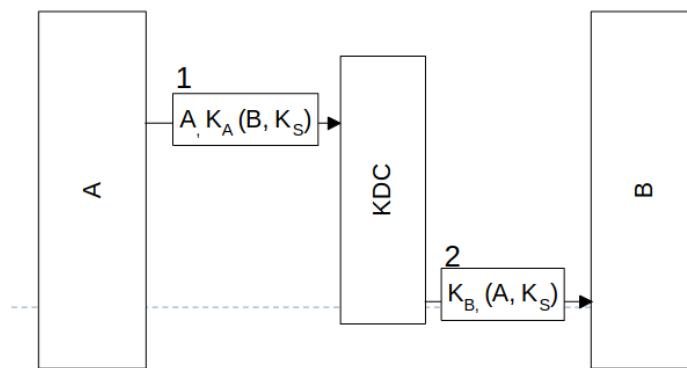


1. Un tercero T simula ser A y envía la identidad de A junto con su challenge R_T .
2. B responde con su challenge R_B y espera la respuesta cifrada.
3. T , no conociendo K_{AB} , inicia una segunda sesión y envía su propio challenge R_B .
4. Cuando B le devuelve $K_{AB}(R_B)$ en la segunda sesión, T lo utiliza como respuesta al primer mensaje, logrando engañar a B y aborta la segunda sesión.

7.4 Reglas de diseño

- El participante que inicia la transmisión debe probar su identidad antes que el receptor.
- Ambos participantes deben usar claves diferentes para la verificación de identidades, incluso si eso implica tener K_{AB} y K'_{AB} .
- Deben elegir challenges de conjuntos diferentes (por ejemplo, A usa pares y B usa impares).
- El protocolo debe resistir ataques con sesiones paralelas.

7.5 Distribución de Claves Confiable (KDC - Key Distribution Center)



1. A selecciona una clave de sesión K_S y comunica al KDC su intención de hablar con B , enviando:

$$A \rightarrow \text{KDC} : K_A(B, K_S)$$

2. El KDC desencripta el mensaje, construye un nuevo mensaje con la identidad de A y la clave de sesión K_S , y lo envía a B encriptado con K_B :

$$\text{KDC} \rightarrow B : K_B(A, K_S)$$

3. B desencripta el mensaje y sabe que A desea comunicarse con él y cuál es la clave de sesión.

8 Ataques de Red

A continuación se presenta una clasificación técnica y concisa de los principales ataques de red, sus objetivos, ejemplos típicos y medidas de protección recomendadas.

Sniffing

- **Descripción:** Escuchar pasivamente el tráfico de la red sin interferir en la conexión.
- **Objetivo:** Descubrir contraseñas, tokens y otra información confidencial transmitida en claro.
- **Ejemplo:** Captura de paquetes en una LAN mediante `tcpdump` o Wireshark.
- **Protección:** Cifrado de los datos en tránsito (por ejemplo TLS, IPsec), uso de redes seguras y segmentación, evitar protocolos que transmitan credenciales en claro.

Spoofing

- **Descripción:** Suplantar la identidad de otro host o usuario interviniendo o forjando una conexión.
- **Objetivo:** Obtener acceso a recursos confiados sin privilegios legítimos.
- **Ejemplo:** Adivinación del número de secuencia TCP para insertar paquetes válidos.
- **Protección:** Autenticación fuerte a nivel de protocolo, uso de canales cifrados, filtrado de paquetes en perímetros (anti-spoofing), validación de números de secuencia y mecanismos como TCP-AO / IPsec.

Hijacking

- **Descripción:** Robar una conexión activa después de que la autenticación se haya completado con éxito.
- **Objetivo:** Acceder a recursos o sesiones sin poseer las credenciales originales.
- **Ejemplo:** Secuestro de sesión web reutilizando cookies o tokens.
- **Protección:** Encriptación de sesión (TLS), renovación de tokens, autenticación multifactor y protección del contexto de sesión.

Ingeniería social

- **Descripción:** Aprovechar la buena voluntad, desconocimiento o descuido de los usuarios para obtener privilegios.
- **Objetivo:** Obtener credenciales o permisos por medio de manipulación humana.
- **Ejemplo:** Envío de un correo aparentando ser `root` solicitando credenciales.
- **Protección:** Autenticación fuerte, educación y entrenamiento a usuarios, procedimientos formales de verificación de identidad y políticas de no divulgación.

Explotación de bugs de software

- **Descripción:** Aprovechar errores de implementación en software o stacks de red.
- **Objetivo:** Obtener acceso no autorizado o ejecutar código arbitrario.
- **Ejemplo:** Vulnerabilidades en servidores HTTP, desbordamientos de búfer en servicios de red.
- **Protección:** Pruebas de seguridad, actualizaciones y parches regulares, uso de listas y avisos de seguridad (CERT), y análisis estático/dinámico del software.

Confianza transitiva

- **Descripción:** Abusar de relaciones de confianza entre usuarios o hosts (por ejemplo, confianza implícita en entornos UNIX o relaciones de confianza entre dominios).
- **Objetivo:** Elevar privilegios o moverse lateralmente dentro de la red.
- **Ejemplo:** Suplantación de dirección IP en entornos que confían en identidades de red.
- **Protección:** Autenticación fuerte, aislamiento y segmentación de red, filtrado estricto de paquetes y políticas de menor privilegio.

Ataques dirigidos por datos

- **Descripción:** Ataques que se activan a partir del procesamiento de datos maliciosos recibidos (payload-driven attacks).
- **Objetivo:** Ejecutar código o alterar comportamiento del sistema al procesar datos maliciosos.
- **Ejemplos:** Código JavaScript malicioso en páginas web, inyección en parsers.
- **Protección:** Validación y saneamiento estricto de entradas, firma y verificación de código/recursos, políticas de contenido (CSP), y sandboxing.

Caballo de Troya

- **Descripción:** Programa legítimo alterado o malicioso que realiza acciones no autorizadas cuando se ejecuta.
- **Objetivo:** Obtener privilegios locales o exfiltrar información.
- **Ejemplo:** Programa de login falso que recopila credenciales.
- **Protección:** Uso de firmas digitales para validar binarios, controles de integridad, políticas de ejecución restringida y concienciación del usuario.

Denegación de Servicio (DoS)

- **Descripción:** Interrumpir o degradar la disponibilidad de servicios para usuarios legítimos.
- **Objetivo:** Impedir el uso de recursos o servicios.
- **Ejemplo:** *Mail bombing*, *ping of death*, ataques volumétricos o amplificados.
- **Protección:** Medidas de mitigación (filtrado, rate limiting, scrubbing), arquitecturas redundantes y uso de proveedores anti-DDoS; no existe una única solución universal.

Enrutamiento fuente

- **Descripción:** Manipulación de la ruta de retorno de paquetes (source routing) o rutas de enrutamiento para interceptar o redirigir tráfico.
- **Objetivo:** Acceder a recursos confiados, redirigir comunicaciones.
- **Protección:** Deshabilitar enrutamiento de origen en routers, filtrado de rutas, y políticas de control de rutas en BGP/infraestructura.

Adivinación de passwords

- **Descripción:** Prueba sistemática (brute force) o dirigida (dictionary, credential stuffing) de contraseñas para acceder a cuentas.
- **Objetivo:** Acceder como usuario legítimo.
- **Ejemplo:** Uso de herramientas automáticas que prueban combinaciones de contraseñas.
- **Protección:** Autenticación multifactor, políticas de contraseñas robustas, detección y bloqueo de intentos de fuerza bruta.

Mensajes de control de red (ICMP, etc.)

- **Descripción:** Aprovechar mensajes de control (p. ej. ICMP) para explotar implementaciones deficientes del stack TCP/IP.
- **Objetivo:** Obtener información de red, redirigir tráfico o forzar errores.
- **Ejemplo:** ICMP `redirect`, mensajes falsos de `Destination Unreachable`.
- **Protección:** Filtrado de mensajes ICMP/ICMPv6 según políticas, endurecimiento de stacks y validación en el plano de control.